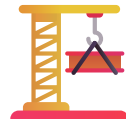


Structures in C Programming



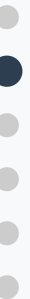
Understanding user-defined data types in C programming

Introduction to Structures

To store numbers, characters, strings, and even large sets of these primitives using arrays, but what if we want to store collections of different kinds of data that are somehow related.

For example, a file about an employee will probably have his/her name, age, hours of work, salary, etc. Physically, all of that is usually stored in someone's filing cabinet. In programming, Let's say you have a group of employees, and you want to make a database! It just wouldn't do to have tons of loose variables hanging all over the place. Then we need to have a single data entity where we will be able to store all related information together. But this can't be achieved by using arrays alone, as in the case of arrays, we can group multiple data elements that are of the same data type, and is stored in consecutive memory locations, and is individually accessed by a subscript.

The Structure is commonly referred to as a user-defined data type. C's structures allow you to store multiple variables of any type in one place (the structure). A structure can contain any of C's data types, including arrays and other structures. Each variable within a structure is called a member of the structure.



They can hold any number of variables, and you can make arrays of structures. This flexibility makes structures ideally useful for creating databases in C. Similar to the structure there is another user-defined data type called Union which allows a programmer to view a single storage in more than one way i.e., a variable declared as union can store within its storage space, the data of different types, at different times.

In this unit, we will be discussing the user-defined data type structures and unions

Declaration of Structures

Arrays allow to define type of variables that can hold several data items of the same kind. Similarly, structure is another user-defined data type available in C that allows to combine data items of different kinds.



Structure Declaration Syntax

```
struct structure-tag  
{  
    datatype variable1;  
    datatype variable2;  
    datatype variable3;  
    ...  
};
```

The structure tag is optional and each member definition is a normal variable definition, such as `int i;` or `float f;` or any other valid variable definition. At the end of the structure's definition, before the final semicolon, you can specify one or more structure variables but it is optional.

Here is the way you would declare a Book structure:



Book Structure Example

```
struct Books
{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
} book;
```

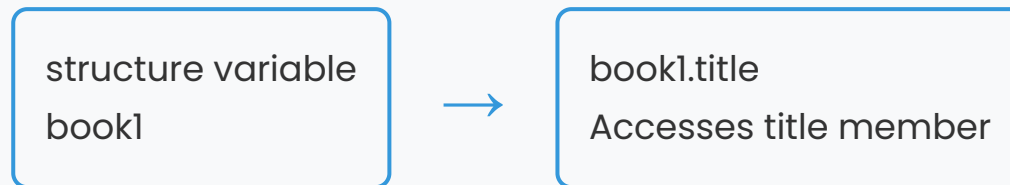
This defines a structure which can be referred to either as struct books or Books, whichever you prefer.

Strictly speaking, you don't need a tag name both before and after the braces if you are not going to use one or the other. But it is a standard practice to put them both in and to give them the same name, but one after the braces starts with an uppercase letter.

The typedef statement doesn't occupy storage: it simply defines a new type. Variables that are declared with the typedef above will be of type struct book.

Accessing Members of a Structure

To access any member of a structure, we use the member access operator (.). The member access operator is coded as a period between the structure variable name and the structure member that we wish to access.



You would use the keyword struct to define variables of structure type.

The following syntax shows how to use a structure:



Coordinate Structure Example

```
struct coordinate
{
    int x;
    int y;
};
```

Thus, to have a structure named first refer to a screen location that has coordinates x=50, y=100, write as:



Accessing Coordinate Members

```
first.x = 50;
first.y = 100;
```

The individual members of the structure behave like ordinary data elements and can be accessed accordingly.

Example Program with Book Structure



Complete Program Example

```
#include<stdio.h>
#include<string.h>
struct Books
{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
};
int main( )
{
    struct Books Book1; /* Declare Book1 of type Book */
    struct Books Book2; /* Declare Book2 of type Book */
    /* book 1 specification */
    strcpy( Book1.title, "C Programming");
    strcpy( Book1.author, "Nuha Ali");
    strcpy( Book1.subject, "C Programming Tutorial");
    Book1.book_id = 6495407;
    /* book 2 specification */
    strcpy( Book2.title, "Telecom Billing");
```



```
strcpy( Book2.author, "Zara Ali");
strcpy( Book2.subject, "Telecom Billing Tutorial");
Book2.book_id = 6495700;
/* print Book1 info */
printf( "Book 1 title : %s\n", Book1.title);
printf( "Book 1 author : %s\n", Book1.author);
printf( "Book 1 subject : %s\n", Book1.subject);
printf( "Book 1 book_id : %d\n", Book1.book_id);
/* print Book2 info */
printf( "Book 2 title : %s\n", Book2.title);
printf( "Book 2 author : %s\n", Book2.author);
printf( "Book 2 subject : %s\n", Book2.subject);
printf( "Book 2 book_id : %d\n", Book2.book_id);
return 0;
}
```

Output of Example Program



Program Output

Book 1 title : C Programming

Book 1 author : Nuha Ali

Book 1 subject : C Programming Tutorial

Book 1 book_id : 6495407

Book 2 title : Telecom Billing

Book 2 author : Zara Ali

Book 2 subject : Telecom Billing Tutorial

Book 2 book_id : 6495700

Explanation



Two Book Variables

Book1 and Book2 are separate variables of type Books



Data Assignment

Each book variable stores different book information



Display Function

printf() is used to display the contents of each book structure